```
#!/usr/bin/env python3
"""
API Security Framework: automated BOLA (Broken Object Level Authorization) testing.

Three layers, one command:
  1. Native two-user BOLA check (user B requests user A's objects)
  2. Auto-generated Postman collection executed with Newman
  3. OWASP ZAP spider / active scan through the ZAP API

Only run against systems you own or are explicitly authorized to test.
"""
import argparse
import difflib
import json
import os
import shutil
import subprocess
import sys
import time
from dataclasses import asdict, dataclass
from pathlib import Path

import requests

SAMPLE_CONFIG = {
    "base_url": "https://api.example.com",
    "openapi_url": "https://api.example.com/openapi.json",
    "users": {
        "alice": {"token": "${ALICE_TOKEN}"},
        "bob": {"token": "${BOB_TOKEN}"},
    },
    "endpoints": [
        {"name": "Get order", "method": "GET", "path": "/orders/{id}",
         "owner": "alice", "resource_id": "1001"},
        {"name": "Get profile", "method": "GET", "path": "/users/{id}",
         "owner": "alice", "resource_id": "42"},
        {"name": "Delete document", "method": "DELETE", "path": "/documents/{id}",
         "owner": "alice", "resource_id": "777", "safe": False},
    ],
    "zap": {"host": "http://127.0.0.1:8080", "api_key": "${ZAP_API_KEY}"},
    "timeout": 15,
}

DENY_CODES = {401, 403, 404}


@dataclass
class Finding:
    source: str
    severity: str
    title: str
    endpoint: str
    detail: str
```

```
# ---------------------------------------------------------------------------- config
def load_config(path: str) -> dict:
    raw = Path(path).read_text(encoding="utf-8")
    return json.loads(os.path.expandvars(raw))


def auth(user: dict | None) -> dict:
    return {"Authorization": f"Bearer {user['token']}"} if user else {}


def build_url(cfg: dict, ep: dict) -> str:
    return cfg["base_url"].rstrip("/") + ep["path"].replace("{id}", str(ep["resource_id"]))


# ---------------------------------------------------------------------------- 1. native
def similar(a: str, b: str) -> float:
    try:
        if json.loads(a) == json.loads(b):
            return 1.0
    except ValueError:
        pass
    return difflib.SequenceMatcher(None, a, b).ratio()


def run_native(cfg: dict, allow_unsafe: bool) -> list[Finding]:
    findings: list[Finding] = []
    timeout = cfg.get("timeout", 15)
    users = cfg["users"]
    for ep in cfg["endpoints"]:
        label = f"{ep['method']} {ep['path']}"
        if ep.get("safe", True) is False and not allow_unsafe:
            findings.append(Finding("native", "Info", "Skipped destructive endpoint", label,
                                    "Re-run with --allow-unsafe on a test environment."))
            continue
        url, method = build_url(cfg, ep), ep["method"]
        owner = users[ep["owner"]]
        base = requests.request(method, url, headers=auth(owner), timeout=timeout)
        if not base.ok:
            findings.append(Finding("native", "Info", "Baseline failed", label,
                                    f"Owner got HTTP {base.status_code}; check resource_id/token."))
            continue
        for name, user in users.items():
            if name == ep["owner"]:
                continue
            r = requests.request(method, url, headers=auth(user), timeout=timeout)
            if r.status_code in DENY_CODES:
                continue
            if r.ok:
                ratio = similar(base.text, r.text)
                sev = "High" if ratio > 0.9 or method != "GET" else "Medium"
                findings.append(Finding(
                    "native", sev, "BOLA: cross-user object access", label,
                    f"'{name}' got HTTP {r.status_code} on {ep['owner']}'s object "
                    f"(response similarity {ratio:.0%})."))
        r = requests.request(method, url, timeout=timeout)
        if r.ok:
            findings.append(Finding("native", "High", "Missing authentication", label,
```

```
                                    f"Unauthenticated request returned HTTP {r.status_code}."))
    return findings


# ---------------------------------------------------------------------- 2. postman
def build_postman(cfg: dict, outdir: Path) -> tuple[Path, Path]:
    items = []
    for ep in cfg["endpoints"]:
        for name in cfg["users"]:
            if name == ep["owner"]:
                continue
            items.append({
                "name": f"{ep['name']}: {name} -> {ep['owner']}'s object",
                "request": {
                    "method": ep["method"],
                    "header": [{"key": "Authorization", "value": f"Bearer {{{{token_{name}}}}}"}],
                    "url": {"raw": "{{base_url}}" + ep["path"].replace("{id}",
str(ep["resource_id"]))},
                },
                "event": [{"listen": "test", "script": {"type": "text/javascript", "exec": [
                    "pm.test('BOLA: access denied for non-owner', function () {",
                    "  pm.expect(pm.response.code).to.be.oneOf([401, 403, 404]);",
                    "});"]}}],
            })
    collection = {
        "info": {"name": "BOLA Test Suite",
                 "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"},
        "item": items,
    }
    env = {"name": "BOLA Env", "values": [{"key": "base_url", "value": cfg["base_url"]}] + [
        {"key": f"token_{n}", "value": u["token"]} for n, u in cfg["users"].items()]}
    c_path, e_path = outdir / "bola.postman_collection.json", outdir /
"bola.postman_environment.json"
    c_path.write_text(json.dumps(collection, indent=2))
    e_path.write_text(json.dumps(env, indent=2))
    return c_path, e_path


def run_newman(cfg: dict, outdir: Path) -> list[Finding]:
    c_path, e_path = build_postman(cfg, outdir)
    if not shutil.which("newman"):
        return [Finding("newman", "Info", "Newman not installed", "-",
                        f"Collection written to {c_path}; install with: npm i -g newman")]
    report = outdir / "newman_report.json"
    subprocess.run(["newman", "run", str(c_path), "-e", str(e_path),
                    "--reporters", "cli,json", "--reporter-json-export", str(report)],
                   check=False)
    if not report.exists():
        return [Finding("newman", "Info", "Newman produced no report", "-", "Check Newman output.")]
    run = json.loads(report.read_text())["run"]
    return [Finding("newman", "High", "Postman BOLA assertion failed",
                    f.get("source", {}).get("name", "?"),
                    f.get("error", {}).get("message", ""))
            for f in run.get("failures", [])]


# ---------------------------------------------------------------------- 3. zap
ACCESS_CWES = {"639", "284", "285", "862", "863", "200"}
```

```
def run_zap(cfg: dict) -> list[Finding]:
    try:
        from zapv2 import ZAPv2
    except ImportError:
        return [Finding("zap", "Info", "zapv2 not installed", "-", "pip install python-owasp-zap-
v2.4")]
    z = cfg.get("zap", {})
    host = z.get("host", "http://127.0.0.1:8080")
    zap = ZAPv2(apikey=z.get("api_key", ""), proxies={"http": host, "https": host})
    base = cfg["base_url"]
    try:
        first_owner = next(iter(cfg["users"].values()))
        zap.replacer.add_rule(description="bola-auth", enabled="true", matchtype="REQ_HEADER",
                              matchregex="false", matchstring="Authorization",
                              replacement=f"Bearer {first_owner['token']}")
        if cfg.get("openapi_url"):
            zap.openapi.import_url(cfg["openapi_url"], hostoverride=base)
        sid = zap.spider.scan(base)
        while int(zap.spider.status(sid)) < 100:
            time.sleep(2)
        aid = zap.ascan.scan(base)
        while int(zap.ascan.status(aid)) < 100:
            time.sleep(5)
        alerts = zap.core.alerts(baseurl=base)
    except Exception as exc:  # ZAP not running / bad API key
        return [Finding("zap", "Info", "ZAP scan failed", "-", str(exc))]
    sev_map = {"High": "High", "Medium": "Medium", "Low": "Low", "Informational": "Info"}
    out = []
    for a in alerts:
        tag = " [access-control related]" if str(a.get("cweid")) in ACCESS_CWES else ""
        out.append(Finding("zap", sev_map.get(a["risk"], "Info"), a["alert"] + tag,
                           f"{a['method']} {a['url']}", a.get("description", "")[:300]))
    return out


# ---------------------------------------------------------------------------- reporting
def write_report(findings: list[Finding], outdir: Path) -> None:
    (outdir / "report.json").write_text(json.dumps([asdict(f) for f in findings], indent=2))
    order = {"High": 0, "Medium": 1, "Low": 2, "Info": 3}
    lines = ["# BOLA Security Report", "", "| Severity | Source | Title | Endpoint |", "|---|---|---
|---|"]
    for f in sorted(findings, key=lambda x: order.get(x.severity, 9)):
        lines.append(f"| {f.severity} | {f.source} | {f.title} | `{f.endpoint}` |")
    (outdir / "report.md").write_text("\n".join(lines))


def main() -> int:
    p = argparse.ArgumentParser(description="Automated BOLA testing: native + Postman/Newman + OWASP
ZAP")
    sub = p.add_subparsers(dest="cmd", required=True)
    sub.add_parser("init", help="write a sample config.json")
    r = sub.add_parser("run", help="run the tests")
    r.add_argument("-c", "--config", default="config.json")
    r.add_argument("-o", "--out", default="reports")
    r.add_argument("--zap", action="store_true", help="also run ZAP spider + active scan")
    r.add_argument("--skip-newman", action="store_true")
```

```
    r.add_argument("--allow-unsafe", action="store_true", help="test non-safe (e.g. DELETE)
endpoints")
    args = p.parse_args()

    if args.cmd == "init":
        Path("config.json").write_text(json.dumps(SAMPLE_CONFIG, indent=2))
        print("Wrote config.json. Set ALICE_TOKEN, BOB_TOKEN, ZAP_API_KEY env vars.")
        return 0

    cfg = load_config(args.config)
    outdir = Path(args.out)
    outdir.mkdir(parents=True, exist_ok=True)

    findings = run_native(cfg, args.allow_unsafe)
    if not args.skip_newman:
        findings += run_newman(cfg, outdir)
    if args.zap:
        findings += run_zap(cfg)

    write_report(findings, outdir)
    high = sum(f.severity == "High" for f in findings)
    print(f"\n{len(findings)} findings, {high} high. Reports in {outdir}/")
    return 1 if high else 0  # non-zero exit fails CI pipelines


if __name__ == "__main__":
    sys.exit(main())
```